

STANDARD OPERATING PROCEDURE

Jinda POS Platform

Complete Operations Guide

Document Version: 1.0
Last Updated: July 2026
Prepared for: Administrators, Developers, and End-Users

Desktop Accounting & POS System for Bhutanese Businesses

Table of Contents

1. Introduction	7
1.1 Purpose of This Document	7
1.2 What is Jinda?	7
1.3 Who Should Read This	7
1.4 Document Conventions	7
1.5 Key Features Overview	8
1.6 Getting Help	8
1.7 Document Version History	9
2. System Overview	9
2.1 Architecture in Simple Terms	9
2.2 Technology Stack (What It Is Built With)	9
2.3 How The Two Parts Talk	10
2.4 Data Storage Summary	10
3. Roles and Responsibilities	10
4. Prerequisites and Requirements	11
4.1 For the Next.js Server (Developer/Admin)	11
4.2 For the POS Desktop App (Developer/Builder)	11
4.3 For the End-User (Shop Computer Running the Built App)	12
5. Environment Setup	12
5.1 Server Environment Variables	12
5.2 Desktop App Environment Variables	13
5.3 MongoDB Setup	14
5.4 Redis Setup (Optional)	14
5.5 Gmail SMTP Setup (Important - emails will not send without this)	14
5.6 Install Server Dependencies	15
5.7 Install Desktop App Dependencies	15
6. Installation Procedures	15
6.1 Seed the Admin User (Server)	15
6.2 Run the Server (Development)	15
6.3 Build and Run the Server (Production)	15
6.4 Install the Desktop App (End-User)	16
6.5 First Launch of the Desktop App	16
7. Configuration Reference	16
7.1 Server `package.json` Scripts	16

7.2 Desktop App `package.json` Scripts	17
7.3 Database Tables (Desktop App SQLite)	17
7.4 Server MongoDB Models	18
7.5 Default Chart of Accounts (Desktop App)	18
7.6 Default Settings (Desktop App)	18

8. Running the System 18

8.1 Starting the Server (Development)	18
8.2 Running the Desktop App in Development	19
8.3 Connecting the Desktop App to Your Local Server (Dev)	19
8.4 Running the Desktop App Against Production	19
8.5 Starting the Electron Admin Wrapper (Optional)	19
8.6 Health Check	20

9. Building and Distributing the Desktop App 20

9.1 Build Prerequisites Checklist	20
9.2 Build the Windows Installer (Main Target)	20
9.3 Build for Mac	20
9.4 Build for Linux	20
9.5 Quick Test Build	20
9.6 Publishing an Update	21
9.7 Build Troubleshooting	21

10. Server Admin Dashboard Operations 21

10.1 Admin Login	21
10.2 Dashboard (`/admin/dashboard`)	21
10.3 License Management	22
10.4 Customer Management (`/admin/customers`)	22
10.5 User (POS Account) Management (`/api/admin/users`)	22
10.6 Update Management (`/admin/updates`)	22
10.7 Messages (`/admin/messages`)	23
10.8 Security Dashboard (`/admin/security`)	23
10.9 Settings (`/admin/settings`)	23
10.10 Full API Endpoint Reference	23

11. Desktop App - First Run and Onboarding 25

11.1 License Agreement (EULA)	25
11.2 Welcome Screen	25
11.3 License Activation Flow (Detailed)	26
11.4 Create the Initial Admin Account	26

11.5 Email Verification (For POS SaaS Accounts)	26
11.6 Device Verification (New Device Login)	26
11.7 Trial Expiry and License Expiry	27
12. Desktop App - Daily Operations	27
12.1 Login	27
12.2 Dashboard (Admin)	27
12.3 POS - Making a Sale	27
12.4 Inventory Management	28
12.5 Contacts (Customers and Suppliers)	28
12.6 Transactions (Receive / Pay / Transfer Money)	28
12.7 Purchase Orders (Admin)	28
12.8 Quotations	29
12.9 Refunds (Admin)	29
12.10 Expenses (Admin)	29
12.11 Recurring Transactions (Admin)	29
12.12 Held Carts	29
12.13 Settings (Admin)	29
12.14 Printers and Printing	30
13. Accounting and GST Operations	30
13.1 Double-Entry Accounting (How It Works Behind The Scenes)	30
13.2 Chart of Accounts	30
13.3 GST (Bhutan)	30
13.4 Filing a GST Return (Step by Step)	30
13.5 Currency	31
13.6 Period Close (Admin)	31
14. Reports and Exports	31
14.1 Reports Page (Admin)	31
14.2 Exporting to Excel	31
14.3 Exporting to PDF	32
14.4 Importing from Excel/CSV (Admin)	32
14.5 Audit Log (Admin)	32
15. HR - Employees and Payroll	32
15.1 Employees (Admin)	32
15.2 Payroll (Admin)	32
15.3 Branches (Admin)	33
16. Backup, Restore, and Cloud Backup	33

16.1 Local Backup (Admin)	33
16.2 Database Encryption at Rest	33
16.3 Cloud Backup (Admin)	33
16.4 Backup Best Practices	33
17. Software Updates	34
17.1 How Auto-Update Works	34
17.2 Installing an Update	34
17.3 Publishing a New Version (Admin)	34
17.4 Rolling Back an Update	34
17.5 Forced Updates	34
18. Security Procedures	34
18.1 Security Overview	34
18.2 Admin Security Duties	35
18.3 Password Policy	35
18.4 Secret Management	36
18.5 What to Do If Compromised	36
19. Troubleshooting	36
19.1 Server Issues	36
19.2 Desktop App Issues	37
19.3 Database Repair Scripts (Python)	37
19.4 Useful Server Scripts	38
19.5 Logs and Diagnostics	39
19.6 Getting Help	39
20. Maintenance Schedule	39
20.1 Daily	39
20.2 Weekly	39
20.3 Monthly	39
20.4 Quarterly	39
20.5 Yearly	40
Appendix A - Command Reference	40
A.1 Server Commands (run in `Jinda POS`)	40
A.2 Desktop App Commands (run in `Jinda POS\New folder\app`)	40
A.3 Useful One-Off Commands	40
A.4 Build Targets	41
Appendix B - File Structure	41
B.1 Server (Root: `Jinda POS`)	41

B.2 Desktop App (^ Jinda POS\New folder\app`) 43

Appendix C - Default Credentials and Locations 45

C.1 Default Credentials 45

C.2 Important File Locations 45

C.3 Runtime Locations (End-User's Computer) 46

C.4 Build Output 47

Appendix D - Glossary 47

Standard Operating Procedure (SOP)

Jinda POS Platform - Complete Operations Guide

Document Version: 1.0

Last Updated: July 2026

Prepared for: Administrators, Developers, and End-Users of the Jinda POS Platform

1. Introduction

1.1 Purpose of This Document

This Standard Operating Procedure (SOP) explains, in simple language, how to **install, configure, run, and maintain** the Jinda POS Platform. It covers both parts of the system:

- The **Next.js Server** (the website and licensing platform running online).
- The **POS Desktop App** (the accounting and point-of-sale software installed on a shop computer).

Follow this document step-by-step. If something is unclear, see the Glossary (Appendix D) for plain-English meanings of technical terms.

1.2 What is Jinda?

Jinda is a **Desktop Accounting & POS (Point of Sale) System built for Bhutanese businesses**. It works fully offline on the shop computer and only connects to the internet for license checks, software updates, and cloud backups.

The platform has two pieces that work together:

Piece	Name	Where it runs	What it does
Server	<code>site-jinda</code>	On the internet (Vercel)	Website, license server, admin dashboard, email, updates
Desktop App	<code>dhisum-tseyig</code> ("Jinda")	On the shop computer	Sales, inventory, accounting, GST, reports, printing

1.3 Who Should Read This

- **Developers** installing and building the software.
- **Server Administrators** managing licenses, customers, and updates.
- **Shop Owners / Cashiers** using the desktop app daily.
- **Accountants** running reports and filing GST.

1.4 Document Conventions

This document uses consistent formatting to help you find information quickly:

- **Bold text** indicates important terms, button names, or menu items (e.g., **Save, Settings**).
- **Code formatting** (monospace font) shows file names, commands, URLs, and technical identifiers (e.g., `package.json`, `npm run dev`).
- **Tables** organize related information for easy comparison.

- **Numbered lists** show step-by-step procedures you must follow in order.
- **Bullet lists** present related items or options without a specific sequence.
- **Callout boxes** highlight important notes, warnings, tips, and security considerations:
- **Note:** Additional information or context.
- **Warning:** Actions that could cause problems if ignored.
- **Tip:** Helpful suggestions or shortcuts.
- **Security:** Critical security practices you must follow.

1.5 Key Features Overview

POS Desktop App Features:

- **Point of Sale:** Fast, intuitive sales interface with barcode scanning, cart management, and multiple payment methods (cash, mBOB, BNB, TPay, credit).
- **Inventory Management:** Track stock levels, set reorder points, manage multiple warehouses, and handle purchase orders.
- **Accounting:** Double-entry bookkeeping with Chart of Accounts, journal entries, and automatic ledger posting.
- **GST Compliance:** Built-in 5% Bhutanese GST support with tax invoices, GST reports, and period-end filing.
- **Customer & Supplier Management:** Maintain contact records with addresses (Dzongkhag/Gewog), credit limits, and transaction history.
- **Reporting:** Comprehensive reports including sales summaries, inventory valuation, aged receivables/payables, profit & loss, and balance sheet.
- **Offline Operation:** Works fully offline with encrypted local database; syncs when internet is available.
- **Printing:** Customizable invoice and receipt templates with thermal printer support.
- **Multi-User:** Role-based access control for cashiers, managers, and administrators.
- **Data Security:** Encrypted SQLite database, secure token storage in OS keychain, and automatic backups.

Server Platform Features:

- **License Management:** Generate, activate, revoke, and track software licenses with device binding.
- **Customer Portal:** Web-based dashboard for customers to view licenses, download updates, and manage their account.
- **Admin Dashboard:** Full administrative control over licenses, customers, users, messages, and security settings.
- **Email System:** Automated emails for license activation, password resets, device verification, and support tickets.
- **Software Updates:** Centralized update distribution with version tracking and rollback capability.
- **API Access:** RESTful API for integration with external systems and automated workflows.

1.6 Getting Help

If you encounter issues or need clarification:

- 1 **Check the Glossary** (Appendix D) for definitions of technical terms.
- 2 **Review the Troubleshooting section** (Section 19) for common problems and solutions.
- 3 **Consult the Command Reference** (Appendix A) for exact command syntax.
- 4 **Contact Support:**

- **Admin Dashboard:** Use the Messages section to submit support tickets.
- **Documentation:** Visit <https://jindapos.com/docs> for additional guides and tutorials.

1.7 Document Version History

Version	Date	Changes
1.0	July 2026	Initial release covering complete platform operations

This document is maintained alongside the software. When significant features are added or procedures change, this SOP will be updated and a new version number assigned. Always use the latest version available from the Jinda POS documentation portal.

2. System Overview

2.1 Architecture in Simple Terms

Think of the platform like a shop with a back office:

- **The Server is the back office.** It keeps the list of who paid for a license, sends emails, hosts the website, and delivers software updates. It runs on the internet so any shop in Bhutan can reach it.
- **The Desktop App is the shop counter.** It does the actual sales, prints bills, tracks stock, and stores all the shop's data safely on that one computer - even when the internet is down.

The desktop app "calls home" to the server to:

- 1 **Activate a license** (first time you enter the license key).
- 2 **Check the license** once a day (silent check).
- 3 **Look for updates** every 4 hours.
- 4 **Back up to the cloud** (if Google Drive or MEGA is set up).

2.2 Technology Stack (What It Is Built With)

Next.js Server (root folder Jinda POS):

- Next.js 16 (website framework)
- MongoDB Atlas (online database for licenses, customers, users)
- Redis (fast cache for rate-limiting; optional, falls back to memory)
- NextAuth (admin login)
- Custom JWT tokens (POS user login)
- Nodemailer (sending emails via Gmail SMTP)
- AWS S3 / Cloudflare R2 (file storage for installers)
- Electron 28 (wraps the website into a desktop admin tool - optional)

POS Desktop App (Jinda POS\New folder\app):

- Electron 32 (turns web code into a desktop program)
- Vite 7 (builds the React interface)
- React 19 (the user interface)
- TypeScript 5.9 (typed JavaScript)
- SQLite via better-sqlite3 (local database file on the shop computer)
- Zustand (manages in-app state)

- shadcn/ui + Tailwind CSS (the look and feel)
- jsPDF + xlsx (PDF and Excel export)
- electron-updater (auto-updates)
- keytar (stores tokens in the OS keychain)

2.3 How The Two Parts Talk

The desktop app sends requests to the server over HTTPS. Only a few allowed websites are accepted (to stop hackers): jindapos.com, api.jinda.com, and dhisum-tseyig.vercel.app. The server exposes a list of **API endpoints** (web addresses that return data). The most important ones are:

- [/api/license/verify](#) - check and activate a license
- [/api/license/activate](#) - bind a license to a device
- [/api/auth/register](#), [/api/auth/login](#) - POS user sign-up and sign-in
- [/api/auth/verify-email](#), [/api/auth/verify-device](#) - email and device verification
- [/api/updates/latest](#) and [/updates/latest.yml](#) - software update info
- [/api/health](#) - health check
- [/api/contact](#) - contact form messages

A full list of all 40 API endpoints is in Section 10 and the Server Admin section.

2.4 Data Storage Summary

Data	Where stored	Example
Licenses, customers, admin users, POS user accounts, updates, audit logs	MongoDB Atlas (online)	License key DTS-XXXX-XXXX-XXXX
Shop sales, inventory, invoices, transactions, employees, payroll, GST entries	SQLite file on the shop computer	%APPDATA%\Jinda\dhisum_tseyig.db
Rate-limiting counters	Redis (online, optional)	"5 license tries per minute"
Update installer files	GitHub Releases / Cloudflare R2	Jinda Setup.exe
POS user session tokens	OS keychain (keytar) on the shop computer	Jinda-POS / saas-tokens

3. Roles and Responsibilities

Role	Who	What they do
Developer	Builds the software	Installs dependencies, runs dev mode, builds installers, fixes bugs

Role	Who	What they do
Server Admin	Runs the online platform	Logs into <code>/admin</code> , creates licenses, manages customers and users, publishes updates, reads security dashboard
Shop Owner (Admin user in desktop app)	Uses the desktop app	Activates license, creates staff users, runs reports, files GST, manages settings and backups
Cashier (Staff user in desktop app)	Uses the desktop app	Makes sales, prints bills, checks stock
Accountant	Reads reports	Uses the Reports and GST pages, exports to Excel/PDF

Access levels in the desktop app:

- **Admin** can do everything (settings, reports, payroll, voiding, backups, user management).
- **Staff** can only use the POS (sales) and related pages. Admin-only pages auto-redirect staff to the POS screen.

4. Prerequisites and Requirements

4.1 For the Next.js Server (Developer/Admin)

Requirement	Details
Node.js	Version 20 or higher
npm	Comes with Node.js
MongoDB Atlas account	Free tier is fine. Create a cluster and get a connection string.
Redis (optional but recommended for production)	Redis Cloud free tier, or Docker <code>redis:7-alpine</code> , or Memurai on Windows
Gmail account	For sending emails (verification, OTP, license transfer). Must enable 2FA and create an App-Specific Password.
Cloudflare R2 or AWS S3 bucket	For hosting installer files (optional if using GitHub Releases)
Vercel account	For hosting the website (free hobby tier works for small use)
OpenRouter API key	For the AI chatbot (optional)

4.2 For the POS Desktop App (Developer/Builder)

Requirement	Details
Operating System	Windows 10/11 (64-bit) for Windows builds. macOS or Linux also supported but Windows is the main target.
Node.js	Version 20 or higher

Requirement	Details
Python	Version 3.x (needed to build the better-sqlite3 native module)
Visual Studio Build Tools	The "C++ build tools" workload (needed to compile native modules like better-sqlite3 and keytar)
Git	To clone/update the repository
Disk space	At least 5 GB free (node_modules + build output)
RAM	8 GB or more (the build uses an 8 GB heap)

4.3 For the End-User (Shop Computer Running the Built App)

Requirement	Details
Operating System	Windows 10/11 (64-bit) recommended. Also runs on macOS and Linux.
RAM	4 GB minimum, 8 GB recommended
Disk space	1 GB free
Printer	Any Windows-compatible printer for A4 invoices; a thermal printer (80mm) for receipts
Internet	Needed only for license activation, daily license check, and updates. The app works fully offline for daily sales.
Scanner	Optional, for barcodes

5. Environment Setup

5.1 Server Environment Variables

Create a file called **.env.local** in the project root **Jinda POS**. Use **.env.example** as a template. Fill in these values:

Variable	What it is	Example
MONGODB_URI	MongoDB Atlas connection string	mongodb+srv://user:pass@cluster.../jinda?...
MONGODB_USE_GOOGLE_DNS	Use Google DNS to fix Atlas DNS issues	true
NEXTAUTH_SECRET	A long random string (min 32 chars) used to sign logins	run openssl rand -base64 32
NEXTAUTH_URL	The website's public URL	https://jindapos.com

Variable	What it is	Example
<code>AWS_ACCESS_KEY_ID</code>	S3/R2 access key	from Cloudflare/AWS
<code>AWS_SECRET_ACCESS_KEY</code>	S3/R2 secret key	from Cloudflare/AWS
<code>AWS_REGION</code>	Region (<code>auto</code> for R2)	<code>auto</code>
<code>AWS_BUCKET_NAME</code>	Bucket name	<code>dhisum-tseyig-downloads</code>
<code>AWS_ENDPOINT_URL</code>	S3 endpoint (R2 custom)	<code>https://<account>.r2.cloudflarestorage.com</code>
<code>SMTP_HOST</code>	Email server host	<code>smtp.gmail.com</code>
<code>SMTP_PORT</code>	Email server port	<code>587</code>
<code>SMTP_USER</code>	Email username (also the From address)	<code>dhisumtseyig@gmail.com</code>
<code>SMTP_PASS</code>	Gmail App-Specific Password (16 chars, no spaces)	from Google 2FA settings
<code>ADMIN_USERNAME</code>	Admin seed username	<code>admin</code>
<code>ADMIN_PASSWORD</code>	Admin seed password (will be hashed)	a strong password
<code>OPENROUTER_API_KEY</code>	For the AI chatbot (optional)	<code>sk-or-v1-...</code>
<code>OPENROUTER_MODEL</code>	AI model name	<code>meta-llama/llama-3.3-70b-instruct:free</code>
<code>JWT_EXPIRY</code>	POS user JWT lifetime	<code>7d</code>
<code>LICENSE_BINDING_SECRET</code>	Secret for license-device binding (optional; defaults to <code>NEXTAUTH_SECRET</code>)	a long random string
<code>REDIS_URL</code>	Redis connection (optional)	<code>redis://localhost:6379</code>

Security note: Never commit the real `.env` or `.env.local` file to git. Only `.env.example` should be shared.

5.2 Desktop App Environment Variables

In `Jinda POS\New folder\app`, create a file called `.env` (use `.env.example` as a template):

Variable	What it is	Example
VITE_API_URL	The server URL the app talks to	Dev: <code>http://127.0.0.1:3000</code> · Prod: <code>https://jindapos.com</code>
GOOGLE_CLIENT_ID	Google OAuth client ID (for Drive backup)	from Google Cloud Console
GOOGLE_CLIENT_SECRET	Google OAuth client secret	from Google Cloud Console
ENCRYPTION_PEPPEP	At least 32-char hex string used to encrypt the local DB and device ID	run <code>openssl rand -hex 32</code>

Important: `VITE_API_URL` is "baked in" at build time. To change the server URL in a built installer, you must rebuild. For dev mode, the app reads `.env` only when not packaged.

5.3 MongoDB Setup

- 1 Go to <https://www.mongodb.com/atlas> and sign up (free).
- 2 Create a cluster (any region close to Bhutan, e.g., AWS Mumbai or GCP Singapore).
- 3 Under **Database Access**, create a database user with a strong password.
- 4 Under **Network Access**, allow your IP (or `0.0.0.0/0` for testing only).
- 5 Click **Connect** -> **Drivers** -> copy the connection string. Replace `<password>` with your user's password. Put it in `MONGODB_URI`.
- 6 Set `MONGODB_USE_GOOGLE_DNS=true` in `.env.local` if you get DNS errors on Windows.

5.4 Redis Setup (Optional)

Redis is used for rate-limiting. If you skip it, the app falls back to in-memory (fine for one server). For production with multiple server instances, use Redis.

Quick local Redis with Docker:

```
docker run -d --name redis -p 6379:6379 redis:7-alpine
```

Then set `REDIS_URL=redis://localhost:6379` in `.env.local`.

Windows alternative: Install Memurai or Redis for Windows.

Production: Use a managed Redis (Redis Cloud, Upstash, or your host's add-on). See `REDIS_SETUP.md` for full details.

5.5 Gmail SMTP Setup (Important - emails will not send without this)

- 1 Go to your Google Account -> **Security** -> enable **2-Step Verification**.
- 2 Go to **App Passwords** (search "app passwords" in Google Account).
- 3 Create an app password for "Mail". You get a 16-character code.
- 4 Copy it **without spaces** into `SMTP_PASS` in `.env.local` (and on Vercel).
- 5 Set `SMTP_HOST=smtp.gmail.com`, `SMTP_PORT=587`, `SMTP_USER=your@gmail.com`.

Without this, verification emails, device OTP emails, and license-transfer OTPs will not arrive. See `FIX-DEVICE-OTP-EMAIL.md` and `FIX-OTP-EMAIL-NOT-ARRIVING.md` for help.

5.6 Install Server Dependencies

In the project root **Jinda POS**:

```
npm install
```

This installs Next.js, MongoDB driver, Redis, NextAuth, Nodemailer, etc.

5.7 Install Desktop App Dependencies

In **Jinda POS\New folder\app**:

```
npm install
```

This also runs **electron-builder install-app-deps** (the **postinstall** script) which rebuilds native modules (**better-sqlite3**, **keytar**) for Electron.

If **better-sqlite3** fails to install: Make sure Python 3 and Visual Studio C++ Build Tools are installed. Run **npm config set msvs_version 2019** if needed. See Troubleshooting (Section 19).

6. Installation Procedures

6.1 Seed the Admin User (Server)

Before the admin dashboard can be used, create the first admin user:

```
npm run seed
```

This reads **ADMIN_USERNAME** and **ADMIN_PASSWORD** from **.env.local**, hashes the password with bcrypt (12 rounds), and saves the admin to MongoDB.

To check the admin user:

```
npx ts-node scripts/check-admin.ts
```

To reset the admin password:

```
npx ts-node scripts/reset-admin.ts <new-password>
```

6.2 Run the Server (Development)

```
npm run dev
```

The server starts at **http://127.0.0.1:3000**. Open it in your browser.

6.3 Build and Run the Server (Production)

```
npm run build  
npm run start
```

For real deployment, use Vercel:

- 1 Push the repository to GitHub.
- 2 Import the project into Vercel.
- 3 Add all environment variables (from Section 5.1) in Vercel's Settings -> Environment Variables.

- 4 Set `NEXTAUTH_URL` to your Vercel URL (e.g., <https://dhisum-tseyig.vercel.app>) or your custom domain (<https://jindapos.com>).
- 5 Deploy. Vercel builds and hosts automatically.

6.4 Install the Desktop App (End-User)

End-users do **not** need Node.js or any developer tools. They just:

- 1 Download **Jinda Setup.exe** from the website's **/download** page (or a direct link you give them).
- 2 Double-click to install. Choose install location (NSIS installer allows custom folder).
- 3 A desktop shortcut "Jinda" is created.
- 4 Launch Jinda from the Start Menu or desktop shortcut.

A **portable** version (**DhisumTseyig-Portable.exe**) is also available - it runs without installation, just extract and run.

6.5 First Launch of the Desktop App

When Jinda opens for the first time:

- 1 The **End-User License Agreement (EULA)** appears. Read and accept it to continue. Acceptance is saved at `%APPDATA%\Jinda\license-agreement.json`. If terms change, the user is asked to accept again.
- 2 The **Welcome** screen appears (no license yet, no trial yet). The user can:
 - **Start a 7-day free trial** (no license key needed), or
 - **Activate a license** (enter a license key bought from the admin).
- 1 After activation, the user creates the **initial admin account** (username + password). This is the local desktop login, stored in the encrypted SQLite database.
- 2 The Dashboard opens. The app is ready to use.

7. Configuration Reference

7.1 Server `package.json` Scripts

Run these in the project root **Jinda POS**:

Command	What it does
<code>npm run dev</code>	Start dev server at <code>http://127.0.0.1:3000</code>
<code>npm run build</code>	Production build
<code>npm run start</code>	Start the production server
<code>npm run start:dev</code>	Start in dev mode
<code>npm run lint</code>	Run ESLint
<code>npm run seed</code>	Create/update the admin user in MongoDB
<code>npm run electron:dev</code>	Launch Electron admin wrapper in dev
<code>npm run electron:build</code>	Build Next.js + Electron admin app

Command	What it does
<code>npm run electron:build:win</code>	Build Windows installer for admin app

7.2 Desktop App `package.json` Scripts

Run these in `Jinda POS\New folder\app`:

Command	What it does
<code>npm run dev</code>	Full dev: Vite renderer + Electron TS watcher + launch Electron
<code>npm run dev:win</code>	Same as <code>dev</code> (Windows)
<code>npm run dev:compile</code>	Compile Electron TypeScript once (no watch)
<code>npm run dev:electron</code>	Launch Electron only (Vite must already be running)
<code>npm run build</code>	Build the React renderer to <code>dist/</code>
<code>npm run lint</code>	Run ESLint
<code>npm run preview</code>	Preview the built renderer
<code>npm run electron:dev</code>	Build + compile + run Electron (dev mode)
<code>npm run electron:start</code>	Build + compile + run Electron (prod mode)
<code>npm run electron:compile</code>	Compile Electron TS to <code>dist-electron/</code> (also writes <code>dist-electron/package.json</code>)
<code>npm run dist</code>	Full distribution build (all platforms)
<code>npm run dist:win</code>	Build Windows NSIS installer + portable
<code>npm run electron:build:win</code>	Alias for <code>dist:win</code>
<code>npm run dist:mac</code>	Build Mac DMG + ZIP
<code>npm run dist:linux</code>	Build Linux AppImage + DEB + RPM
<code>npm test</code>	Run the custom test suite

7.3 Database Tables (Desktop App SQLite)

The desktop app creates these tables automatically on first run. You do not need to create them manually.

Core tables: `users`, `companies`, `accounts` (Chart of Accounts), `contacts` (customers/suppliers), `items` (inventory), `transactions`, `transaction_lines` (double-entry), `stock_movements`, `invoices`, `invoice_items`, `gst_entries`, `audit_logs`, `settings`, `period_locks`.

Feature tables: `purchase_orders`, `purchase_order_items`, `quotations`, `quotation_items`, `held_carts`, `held_cart_items`, `refunds`, `refund_items`, `expenses`, `recurring_transactions`, `recurring_execution_log`, `branches`, `employees`, `payroll_payments`, `barcode_mappings`, `price_lists`, `price_list_items`,

`integrity_hashes`, `item_categories`, `item_units`.

7.4 Server MongoDB Models

The server stores these collections in MongoDB:

Model	Purpose
<code>Admin</code>	Admin dashboard login (NextAuth)
<code>PosUser</code>	POS end-user account (SaaS login, device verification, sessions)
<code>License</code>	Software license (key, plan, device binding, expiry, transfers)
<code>Customer</code>	Customer profile (name, email, company, address)
<code>Session</code>	Refresh-token sessions with reuse detection
<code>Token</code>	Verification/OTP tokens (email verify, reset, license OTP, device OTP)
<code>Trial</code>	Trial tracking per device
<code>Update</code>	Software update release metadata
<code>AuditLog</code>	Security audit trail (29 event types)
<code>ContactMessage</code>	Contact form submissions

7.5 Default Chart of Accounts (Desktop App)

The desktop app seeds a standard Bhutanese chart of accounts on first run:

- **1000 Assets** -> 1100 Cash on Hand, 1200 Bank, 1300 Debtors, 1400 Inventory, 1500 GST Input
- **2000 Liabilities** -> 2100 Creditors, 2200 GST Output, 2300 GST Payable, 2400 PF Payable, 2500 TDS Payable
- **3000 Equity** -> 3000 Capital, 3100 Retained Earnings, 3200 Drawings
- **4000 Income** -> 4000 Sales Revenue, 4100 Other Income
- **5000 Cost of Goods Sold**
- **6000 Operating Expenses** -> 6100 Rent, 6200 Salary, 6300 Utilities, 6400 Other

7.6 Default Settings (Desktop App)

- Company name: "My Business"
- Default GST rate: 5%
- Domestic GST rate: 0%
- Invoice prefix: `DT`
- Default admin login (after first-run setup): the user chooses this. A common test default is `admin / admin123`.

8. Running the System

8.1 Starting the Server (Development)

- 1 Make sure `.env.local` is filled in (Section 5.1).

- 2 Make sure MongoDB Atlas is reachable (check Network Access).
- 3 Make sure the admin user is seeded (`npm run seed`).
- 4 Run:

```
npm run dev
```

- 1 Open `http://127.0.0.1:3000` in your browser.
- 2 The website loads. The admin dashboard is at `http://127.0.0.1:3000/admin/login`.

8.2 Running the Desktop App in Development

You need **two** terminals (or use `npm run dev` which runs both at once).

Option A - All-in-one (recommended):

```
cd "Jinda POS\New folder\app"  
npm run dev
```

This starts Vite (the renderer at `http://127.0.0.1:5173`), compiles Electron TypeScript in watch mode, and launches Electron pointing at the Vite server.

Option B - Step by step:

Terminal 1:

```
cd "Jinda POS\New folder\app"  
npm run dev:compile # compiles electron TS in watch mode  
vite                # starts the renderer
```

Terminal 2 (after Vite is ready):

```
npm run dev:electron
```

8.3 Connecting the Desktop App to Your Local Server (Dev)

In `Jinda POS\New folder\app\.env`, set:

```
VITE_API_URL=http://127.0.0.1:3000
```

Then run the dev build. The desktop app will call your local server for license checks.

8.4 Running the Desktop App Against Production

In `.env`, set:

```
VITE_API_URL=https://jindapos.com
```

Then rebuild the installer (`npm run dist:win`). Distribute the built installer to end-users.

8.5 Starting the Electron Admin Wrapper (Optional)

The root project can also be wrapped in Electron (an admin desktop tool). This is optional - most admins just use the website in a browser.

```
npm run electron:dev # dev mode  
npm run electron:build:win # build Windows installer
```

8.6 Health Check

The desktop app calls `/api/health` to check if the server is alive. You can test it manually:

```
curl https://jindapos.com/api/health
```

Expected response: `{ "status": "ok", "timestamp": "..."}`

9. Building and Distributing the Desktop App

9.1 Build Prerequisites Checklist

Before building the Windows installer, confirm:

- [] Node.js 20+ installed
- [] Python 3 installed and in PATH
- [] Visual Studio Build Tools (C++ workload) installed
- [] `npm install` completed successfully in `New folder\app`
- [] `better-sqlite3` native module compiled (the `postinstall` script does this)
- [] `.env` has the correct `VITE_API_URL` for production
- [] `build/icon.ico` exists (the app icon)
- [] `build/license.txt` exists (the NSIS license text shown during install)

9.2 Build the Windows Installer (Main Target)

```
cd "Jinda POS\New folder\app"
npm run dist:win
```

This does three things:

- 1 `vite build` -> builds the React interface to `dist/`
- 2 `electron:compile` -> compiles Electron TypeScript to `dist-electron/` and writes `dist-electron/package.json`
- 3 `electron-builder --win` -> packages everything into installers in `release/`

Output in `release/`:

- `Jinda Setup.exe` - NSIS installer (allows custom install location, creates shortcuts)
- `DhisumTseyig-Portable.exe` - portable version (no install needed)
- `win-unpacked/Jinda.exe` - unpacked app folder (for testing)

9.3 Build for Mac

```
npm run dist:mac
```

Output: DMG and ZIP in `release/`. Needs a Mac to build (or a macOS VM). Icon: `build/icon.icns`.

9.4 Build for Linux

```
npm run dist:linux
```

Output: AppImage, DEB, RPM in `release/`.

9.5 Quick Test Build

There is a minimal config `electron-builder-simple.json` (applied `com.dhisumtseyig.app`, `productName` `DhisumTest`, output `release-test/`). Use it for quick builds:

```
npx electron-builder --config electron-builder-simple.json --win
```

9.6 Publishing an Update

Once you have a new build, you publish it so existing users get the auto-update:

- 1 Upload `Jinda Setup.exe` to GitHub Releases (or Cloudflare R2).
- 2 Log into the admin dashboard at `/admin/updates/create`.
- 3 Fill in: version number, release notes, download URL, file size, SHA512 hash, rollout %, forced flag.
- 4 Set status to **published**.
- 5 Existing desktop apps will detect the update within 4 hours, download it, and (if the user is admin) install on next restart.

To compute the SHA512 hash on Windows:

```
certutil -hashfile "Jinda Setup.exe" SHA512
```

9.7 Build Troubleshooting

- **better-sqlite3 build fails:** Install VS Build Tools C++ and Python 3. Run `npm rebuild better-sqlite3` or `npm run postinstall`.
- **Electron download is slow:** Set `ELECTRON_MIRROR=https://npmmirror.com/mirrors/electron/` or use the `.electron-cache` folder.
- **Build is out of memory:** The `dev` script already sets `--max-old-space-size=8192`. For builds, close other heavy apps.
- **DB is locked:** Make sure no other Jinda instance is running during build.

10. Server Admin Dashboard Operations

The admin dashboard is at `/admin` on the server (e.g., `https://jindapos.com/admin`). Login with the admin username and password (created by `npm run seed`).

10.1 Admin Login

- 1 Go to `https://jindapos.com/admin/login` (or `http://127.0.0.1:3000/admin/login` in dev).
- 2 Enter the admin username and password.
- 3 You are redirected to `/admin/dashboard`.

Only users with `role === 'admin'` can access admin pages. Non-admins are redirected to the login page. All admin routes are also hidden from search engines via the `X-Robots-Tag: noindex` header set in `middleware.ts`.

10.2 Dashboard (`/admin/dashboard`)

Shows:

- License counts (active, inactive, expired, suspended, revoked)
- Plan breakdowns (Starter, Growth, Pro, Enterprise, Lifetime)

- Licenses expiring soon
- Recent licenses created
- Statistics come from `/api/stats`

10.3 License Management

Create a License (`/admin/licenses/create`)

- 1 Click **Create License**.
- 2 Fill in: customer name, email, company name, plan, max users (auto-set by plan), expiry date.
- 3 Submit. The system generates a unique license key in the format `DTS-XXXX-XXXX-XXXX`.
- 4 Give the license key to the customer. They will enter it in the desktop app to activate.

Plan -> max users mapping:

- Starter = 1 user
- Growth = 2 users
- Pro = 2 users
- Enterprise = 5 users
- Lifetime = 5 users (no expiry)

View / Edit / Delete a License (`/admin/licenses` and `/admin/licenses/[id]`)

- List all licenses with search and pagination.
- Click a license to see details.
- Edit: change customer, plan, expiry, status.
- Delete: removes the license (use with caution).

Extend a License (`/api/admin/licenses/extend`)

Adds N days to the expiry date. If the license was expired, it is reactivated. Used when a customer pays for more time.

Revoke a License (`/api/license/revoke`)

Admin-only. Sets the license status to `revoked` with a reason. Reasons: `payment_failed`, `fraud`, `user_request`, `admin_action`, `terms_violation`. The revoked desktop app will be denied on its next license check.

10.4 Customer Management (`/admin/customers`)

- List all customers (with their license count).
- Create a customer (name, email, company, phone, address with dzongkhag/gewog).
- View a customer with all their licenses.
- Edit or delete a customer (blocked if they have active licenses).

10.5 User (POS Account) Management (`/api/admin/users`)

- List all POS users (paginated).
- Actions per user: `activate`, `suspend`, `terminate`, `disable`, `lock`, `unlock`, `reset_password`, `verify_email`.
- These affect the POS user's ability to log in from the desktop app.

10.6 Update Management (`/admin/updates`)

- **List updates:** See all software releases.

- **Create update:** Add a new version (version, notes, download URL, size, SHA512, rollout %, forced flag, status).
- **Edit update:** Change any field, including status (draft -> published -> blocked -> rollback).
- **Statuses:** **draft** (not live), **published** (live, respects rollout %), **blocked** (paused), **rollback** (rolled back).
- **Rollout %:** Allows gradual rollout (e.g., 25% of users get it first).
- **Forced flag:** If true, users must update before they can continue.

The public endpoints are:

- **/api/updates/latest** - returns the latest published update as JSON (called by desktop apps).
- **/updates/latest.yml** - electron-updater-compatible YAML feed.

10.7 Messages (/admin/messages)

- Inbox of contact form submissions from the website.
- Status: Unread -> Read -> Replied.
- Reply to a message: sends an email to the customer via Nodemailer.

10.8 Security Dashboard (/admin/security)

Shows security metrics from **/api/admin/security/dashboard**:

- Recent security alerts (login failures, token reuse, brute force, suspicious activity)
- Login statistics (success/failure counts)
- Active sessions
- Locked accounts
- Top failing IPs
- Suspicious users
- All events are also written to the **AuditLog** MongoDB collection (29 event types, 3 severity levels).

10.9 Settings (/admin/settings)

Admin settings for the dashboard (branding, etc.). Changes here do not affect desktop app settings.

10.10 Full API Endpoint Reference

Auth:

Endpoint	Methods	Purpose
/api/auth/[...nextauth]	GET, POST	NextAuth admin login handler
/api/auth/register	POST	Register a POS user
/api/auth/login	POST	POS user login (rate-limited, lockout after 5 fails)
/api/auth/logout	POST, GET	Revoke session(s)
/api/auth/refresh	POST	Rotate refresh token (detects reuse)
/api/auth/status	GET	Check if email is verified
/api/auth/verify-email	POST, GET	Verify email via token

Endpoint	Methods	Purpose
/api/auth/resend-verification	POST	Resend email verification link
/api/auth/verify-device	POST	Verify device OTP (6-digit)
/api/auth/resend-device-code	POST	Resend device OTP
/api/auth/verification-status	GET, POST	Poll verification status
/api/auth/forgot-password	POST	Request password reset email
/api/auth/reset-password	POST, GET	Reset password / validate token
/api/auth/change-password	POST	Change password (authenticated)
/api/auth/captcha	GET, POST	Generate/verify captcha

License:

Endpoint	Methods	Purpose
/api/license/verify	POST, GET	Validate + activate license, device transfer
/api/license/activate	POST	Activate license (bind device)
/api/license/revoke	POST, GET	Admin: revoke a license

Admin:

Endpoint	Methods	Purpose
/api/admin/licenses	GET, POST, PATCH	List/create/update licenses
/api/admin/licenses/[id]	GET, DELETE	Get/delete a license
/api/admin/licenses/extend	POST	Extend license expiry
/api/admin/customers	GET, POST	List/create customers
/api/admin/customers/[id]	GET, PATCH, DELETE	Get/update/delete customer
/api/admin/users	GET, PATCH	List/act on POS users
/api/admin/messages	GET, PATCH	List/update messages
/api/admin/messages/[id]/reply	POST	Reply to a message by email
/api/admin/security/dashboard	GET	Security metrics

Updates:

Endpoint	Methods	Purpose
/api/updates	GET, POST	List/create updates

Endpoint	Methods	Purpose
<code>/api/updates/[id]</code>	GET, PUT, PATCH, DELETE	Manage a single update
<code>/api/updates/latest</code>	GET	Public: latest update info
<code>/api/updates/upload</code>	POST	Admin: add update from GitHub URL

Other:

Endpoint	Methods	Purpose
<code>/updates/latest.yml</code>	GET	electron-updater YAML feed
<code>/api/download</code>	GET	Download URLs for installer
<code>/api/health</code>	GET	Health check
<code>/api/contact</code>	POST	Contact form submission
<code>/api/chat</code>	POST	AI chatbot proxy (OpenRouter)
<code>/api/terms</code>	GET	Terms of Service JSON
<code>/api/privacy</code>	GET	Privacy Policy JSON
<code>/api/stats</code>	GET	Dashboard statistics

11. Desktop App - First Run and Onboarding

This section describes what a shop owner sees the first time they open Jinda.

11.1 License Agreement (EULA)

On first launch, the End-User License Agreement appears. The text is fetched from the server (`/api/terms` and `/api/privacy`). The user must click **Accept** to continue. Acceptance is saved locally at `%APPDATA%\Jinda\license-agreement.json`. If the terms version changes, the user must accept again.

11.2 Welcome Screen

If there is no license and no trial yet, the Welcome screen appears. The user has two choices:

A) Start a 7-day free trial:

- No license key needed.
- The app writes `trial.json` and an encrypted `.trial-lock` (both bound to the device ID to prevent resetting the trial by deleting files).
- After 7 days, the trial expires. The user must activate a license to continue.

B) Activate a license:

- Enter the license key (format `DTS-XXXX-XXXX-XXXX`).
- The app sends the key + device ID to the server's `/api/license/verify`.
- The server binds the license to this device and returns an `activationSecret` (only on first activation).

- The license is saved locally in `license.json`.

11.3 License Activation Flow (Detailed)

- 1 User enters the license key.
- 2 App calls `/api/license/verify` with `{licenseKey, deviceId, deviceInfo}`.
- 3 **First activation (no device bound yet):** Server binds the device, returns `valid: true + activationSecret`. App saves `license.json`.
- 4 **Same device, valid activationSecret:** Server increments `activationCount`, returns `valid: true`. Normal re-handshake.
- 5 **Different device (transfer):**
 - Server checks the **monthly transfer limit** (max 2 transfers per month).
 - If the user provided a **password**, the server verifies it (bcrypt) and sends a 6-digit **OTP** to the license holder's email. App asks for the OTP.
 - If the user provided the **OTP**, the server verifies it and rebinds the license to the new device. The `activationSecret` is rotated.
 - If neither, the server returns `CREDENTIALS_REQUIRED` with a masked email hint.
- 1 **Expired or suspended license:** Server returns `valid: false` with a reason. The app shows the appropriate message.

11.4 Create the Initial Admin Account

After successful activation (or trial start), the user creates the first local admin account:

- Username (must be unique)
- Password (must meet the strong password policy: min 8 chars, upper + lower + digit + special)
- This account is stored in the `users` table (bcrypt-hashed) and is sealed by the IntegrityService (tamper-evident).

After this, the Dashboard opens. The initial admin can create more users (staff) from Settings -> Users, up to the plan's seat limit.

11.5 Email Verification (For POS SaaS Accounts)

If the user registered a POS account through the website (not just local desktop login):

- 1 After registration, a verification email is sent with a link.
- 2 The link points to `{NEXTAUTH_URL}/verify-email?token=...`
- 3 Clicking it verifies the email and creates a session.
- 4 If the email does not arrive, see `FIX-OTP-EMAIL-NOT-ARRIVING.md` (SMTP not configured on the server).

11.6 Device Verification (New Device Login)

When a POS user logs in from a new device (different fingerprint):

- 1 After password success, the server detects a new device.
- 2 A 6-digit OTP is emailed to the user.
- 3 The user enters the OTP in the desktop app's Device Verification screen.
- 4 On success, the device is added to `approvedDevices` and a session is created.
- 5 3 wrong OTP attempts -> 15-minute logout.
- 6 Resend is rate-limited (1 per 30 seconds).

11.7 Trial Expiry and License Expiry

- **Trial expired:** The app shows a screen to activate a license. Data is preserved.
- **License expired:** The app shows a screen to renew. The server marks the license **expired** automatically when the expiry date passes.
- **License device mismatch:** The license was bound to another device. The user must transfer it (Section 11.3).
- **Verification required:** The app has not verified with the server for more than 7 days (offline grace). It will ask to connect to the internet.

12. Desktop App - Daily Operations

This section covers the daily use of Jinda at the shop.

12.1 Login

- Open Jinda.
- Enter your username and password.
- Admin users see the Dashboard. Staff users are taken straight to the POS.

12.2 Dashboard (Admin)

Shows real-time metrics:

- Today's sales, GST collected, transaction count
- Low-stock alerts
- Recent transactions
- Header alerts (notifications)

12.3 POS - Making a Sale

The POS is the main screen for cashiers.

- 1 Click **POS** in the sidebar.
- 2 Search for items by name, code, or **barcode scan**.
- 3 Click items to add to the cart. Enter quantity.
- 4 The cart shows: item, price, quantity, GST per line, line total.
- 5 GST is auto-calculated at the rate on the item (default 5%, or the domestic rate if "domestic" tax type is selected).
- 6 Apply a discount if needed.
- 7 Choose a customer (optional - needed for credit sales).
- 8 **Held carts:** You can park a sale (hold) and resume it later. Useful when a customer steps away.
- 9 **Split payments:** Pay with multiple methods (e.g., half cash, half card).
- 10 Payment modes: Cash, Bank, Credit, Card, UPI, mBOB, BNB, TPay, DrukPNB, BDBL, DKBank.
- 11 **Credit control:** If a customer is over their credit limit, the sale is blocked.
- 12 Click **Complete Sale**.
- 13 Choose to print:
 - **A4 invoice** (laser/inkjet printer) - templates: Standard, Modern, Professional
 - **Thermal receipt** (80mm thermal printer)
- 1 Optionally **email the invoice** to the customer (opens the mail client).

2 The sale:

- Reduces inventory (stock movements).
- Updates the customer's ledger (if credit).
- Creates a double-entry transaction (Debit Cash/Bank, Credit Sales Revenue + GST Output).
- Writes a GST entry (output GST).
- Writes an audit log entry.

12.4 Inventory Management

- Click **Inventory** in the sidebar.
- **Add item:** code, name, category, unit, purchase price, selling price, opening stock, reorder level, GST applicable, GST rate.
- **Update item:** change details, prices, stock.
- **Add stock:** increases quantity (records a stock movement "in").
- **Average cost:** automatically recalculated when you add stock at a new price.
- **Low stock:** items below reorder level are flagged. Shown on the Dashboard too.
- **Categories and units:** create custom categories (e.g., "Groceries") and units (e.g., "kg", "piece").
- **Barcodes:** map barcodes to items in the Barcode page. Then scanning at the POS finds the item.

12.5 Contacts (Customers and Suppliers)

- Click **Contacts** in the sidebar.
- **Customers** (**type=customer**): name, contact person, phone, email, address (street, gewog, dzongkhag), credit limit, credit days, opening balance, GST number.
- **Suppliers** (**type=supplier**): same fields.
- Each contact gets an **auto-ledger** (account in the Chart of Accounts) so their balances are tracked automatically.
- **Customer statement:** view a customer's full ledger (invoices, payments, balance) from the Customer Statement page.
- **Tiered pricing:** assign a price list to a customer (special prices per item).

12.6 Transactions (Receive / Pay / Transfer Money)

- Click **Transactions** in the sidebar.
- **Receive Money:** money in from a customer (reduces their receivable). Choose account (Cash, Bank), customer, amount, payment mode.
- **Pay Money:** money out to a supplier (reduces their payable).
- **Transfer:** move money between accounts (e.g., Cash -> Bank).
- **Void:** admin can void a transaction (with a reason). Voiding creates a reversal journal entry. Voided transactions are excluded from reports and GST.
- **Export:** export transactions to Excel.

12.7 Purchase Orders (Admin)

- Click **Purchase Orders** in the sidebar.
- **Create PO:** select supplier, add items with quantity and purchase price. You can also set a **selling price** (PO-driven pricing) that updates the item's selling price on receiving.
- **Receive PO:** when goods arrive, mark the PO as received. Stock increases, average cost updates.
- **Payment:** pay the supplier (creates a transaction).

- Statuses: draft -> ordered -> received -> paid.

12.8 Quotations

- Click **Quotations** in the sidebar.
- **Create quotation:** select customer, add items, quantities, prices. GST auto-calculated.
- **Convert to sale:** one click turns a quotation into a POS sale.
- Statuses: draft -> sent -> accepted -> converted.

12.9 Refunds (Admin)

- Click **Refunds** in the sidebar.
- **Create refund:** select the original invoice, choose items to refund, reason.
- Refund creates a reversal journal entry and a GST adjustment.
- Refunded stock returns to inventory.
- Refunds can be voided (soft delete).

12.10 Expenses (Admin)

- Click **Expenses** in the sidebar.
- **Add expense:** date, category, amount, payment mode, vendor, description, receipt path.
- Each expense creates a transaction (Debit Expense, Credit Cash/Bank).
- **Summary:** view total expenses by category and period.

12.11 Recurring Transactions (Admin)

- Click **Recurring** in the sidebar.
- **Create recurring transaction:** frequency (daily, weekly, monthly, quarterly, yearly), amount, accounts, description.
- The app processes due recurring transactions **hourly** and on startup. The renderer is notified via **recurring:processed** IPC events.

12.12 Held Carts

- In the POS, you can **hold** a cart (park a sale).
- Held carts are saved in the **held_carts** table.
- Resume, load, or delete held carts from the POS held-carts panel.

12.13 Settings (Admin)

Click **Settings** in the sidebar. Sections:

- **Company info:** name, trade license no, tax no (GST), address, phone, email, default GST rate, domestic GST rate, invoice prefix, financial year start.
- **Logo / Seal / Signature:** upload images (PNG/JPEG, validated by magic bytes; logo ≤5MB, seal/signature ≤2MB). Stored as base64 data URIs. Shown on printed invoices.
- **Users:** create staff users (up to plan limit), change passwords. The admin seat is free; only active non-admin staff count against the limit.
- **GST rates:** both standard (5%) and domestic (0%) - both are integrity-sealed (tamper-evident).
- **Period close:** lock a month/year so no transactions can be added or edited for that period. Locks are integrity-sealed.
- **Backup:** create/restore local backups.

- **Cloud backup:** configure Google Drive or MEGA.
- **EULA:** view the license agreement status.

12.14 Printers and Printing

- The app uses the **native Windows print dialog** (Chromium's print preview is disabled).
- **A4 invoices:** laser/inkjet printers. Templates: Standard, Modern, Professional. Bhutanese-themed (crimson, saffron, gold, navy, prayer-flag gradient).
- **Thermal receipts:** 80mm thermal printers. Compact layout.
- **Reports:** printed via hidden BrowserWindow -> `webContents.print()`.
- **Barcodes:** printed as label sheets.
- All printed HTML is **sanitized** (scripts, iframes, event handlers stripped) for security.
- `getPrinters()` returns a stub list. Use the Windows print dialog to pick the real printer.

13. Accounting and GST Operations

13.1 Double-Entry Accounting (How It Works Behind The Scenes)

Every sale, purchase, payment, receipt, transfer, expense, and refund creates a **transaction** with **transaction_lines** (debits and credits). The system ensures debits = credits for every transaction. This is double-entry bookkeeping, done automatically.

Example - a cash sale of Nu. 100 + 5% GST:

- Debit Cash on Hand (1100) - Nu. 105
- Credit Sales Revenue (4000) - Nu. 100
- Credit GST Output (2200) - Nu. 5

13.2 Chart of Accounts

Pre-seeded (see Section 7.5). Admin can add custom accounts from the Transactions page (account type, subtype, parent account).

13.3 GST (Bhutan)

- **Default GST rate:** 5% (standard).
- **Domestic GST rate:** 0% (for B2B/domestic transactions; set in Settings).
- **GST Input:** GST paid on purchases (account 1500).
- **GST Output:** GST collected on sales (account 2200).
- **GST Payable:** net GST owed to the government (account 2300) = Output – Input.

GST Page (Admin):

- **Summary:** for a given month/year, shows taxable sales, taxable purchases, GST output, GST input, exempt sales/purchases, net GST payable.
- **Returns:** list of filed periods.
- **File return:** marks a month/year as filed (admin). Prevents changes to that period's GST entries.
- All voided transactions are excluded. Refunds create GST adjustments.

13.4 Filing a GST Return (Step by Step)

- 1 At the end of the month, go to **GST** in the sidebar.
- 2 Select the month and year.

- 3 Review the summary: total sales, total purchases, GST output, GST input, net payable.
- 4 Click **File Return** (admin only).
- 5 The period is marked as filed. The **gst_entries** for that period are marked **is_filed**.
- 6 Print the GST summary report (A4) for your records or for filing with the Department of Revenue.

13.5 Currency

All amounts are in **Bhutanese Ngultrum (Nu.)**, formatted as **Nu. 1,234.56**.

13.6 Period Close (Admin)

To prevent edits to a closed period:

- 1 Go to **Settings -> Period Close**.
- 2 Select the year and month.
- 3 Click **Lock**.
- 4 The **period_locks** entry is sealed (integrity-protected). No transactions can be added or edited for that period.

14. Reports and Exports

14.1 Reports Page (Admin)

Click **Reports** in the sidebar. Available reports:

Report	What it shows
Trial Balance	All accounts with their debit/credit balances (must balance)
Profit & Loss	Income vs. expenses for a period -> net profit/loss
Balance Sheet	Assets, liabilities, equity as of a date
Outstanding	Who owes you (receivables) and who you owe (payables)
Stock Report	Items, quantities, value (at average cost), low/out of stock
Sales Report	Sales by period, with summary cards (revenue, GST, count)
Payroll Report	Salary payments by period
Purchase Report	Purchases by period
Customer Insights	Per-customer sales, balances, top customers
Aged Receivables	Receivables by age bucket (0 - 30, 31 - 60, 61 - 90, 90+ days)
Aged Payables	Payables by age bucket

14.2 Exporting to Excel

Almost every list and report can be exported to Excel (.xlsx):

- Sales, purchases, transactions, contacts, items, expenses, employees, quotations, refunds, aged reports.

- Excel files are styled with the Bhutanese theme (crimson/saffron headers, alternating cream rows, currency format).
- Uses the `xlsx` library (SheetJS).

14.3 Exporting to PDF

Reports can also be exported to PDF:

- Uses `jsPDF + jsPDF-autotable`.
- Same Bhutanese theme (prayer-flag bar, crimson header, page numbers, "Powered by PhoJaa95" footer).
- Good for printing or emailing reports.

14.4 Importing from Excel/CSV (Admin)

- Go to **Import/Export** in the sidebar.
- **Import contacts:** upload an Excel/CSV with customer or supplier data (name, contact person, phone, email, address, GST number, credit limit, credit days, opening balance).
- **Import items:** upload an Excel/CSV with inventory data (code, name, category, unit, prices, stock, GST).
- Bounded to 10,000 rows per import.
- Sample files are included in the project root: `Employees.xlsx`, `Transactions.xlsx`.

14.5 Audit Log (Admin)

- Click **Audit Log** in the sidebar.
- Shows every action (create, update, delete, void, login) with user, entity, old/new values, IP, timestamp.
- The audit log is **tamper-evident** (linked chain verified on every boot).
- Useful for tracking who did what and when.

15. HR - Employees and Payroll

15.1 Employees (Admin)

- Click **Employees** in the sidebar.
- **Add employee:** employee no, name, position, department, phone, email, salary, join date, PF rate, GIS amount, TDS rate, HC (health contribution) rate.
- **Export:** export all employees to Excel (`Employees.xlsx`).

15.2 Payroll (Admin)

- Click **Payroll** (accessed from the Employee page or sidebar).
- **Process payroll:** for a month/year, the system calculates:
 - Gross salary
 - PF (Provident Fund) deduction
 - TDS (Tax Deducted at Source)
 - GIS (Group Insurance Scheme)
 - HC (Health Contribution)
 - Net salary

- **Pay:** creates a payment transaction (Debit Salary Expense, Credit Cash/Bank + liability accounts for deductions).
- **History:** view past payroll runs per employee.

15.3 Branches (Admin)

- Click **Branches** in the sidebar.
- **Add branch:** name, code (unique), address, phone, email.
- For businesses with multiple locations.

16. Backup, Restore, and Cloud Backup

16.1 Local Backup (Admin)

- Go to **Settings -> Backup**.
- **Create backup:** exports the entire database to a JSON file (streaming, memory-bounded even for large DBs).
- **Restore backup:** imports a JSON backup file. The app **relaunches** after restore.
- **Auto backup:** runs daily at 23:00 automatically. Status shown in `backup:autoBackupStatus`.
- **Crash recovery:** if the database fails to open, the app tries the newest `backups/emergency_*.db` file (decrypting if needed).

16.2 Database Encryption at Rest

- When Jinda is **running**, the database is a plaintext file `dhisum_tseyig.db`.
- When Jinda **closes** (before-quit), the database is **encrypted** to `dhisum_tseyig.db.secure` using AES-256-CBC + HMAC, and the plaintext file is **securely deleted**.
- When Jinda **opens**, it decrypts `.secure` back to `.db`.
- This protects the shop's data if the computer is stolen.
- The encryption uses the `ENCRYPTION_PEPPER` and a per-install secret (`.install-key`).

16.3 Cloud Backup (Admin)

- Go to **Settings -> Cloud Backup**.
- **Google Drive:** connect via OAuth (a loopback server on port 38291 with a CSRF nonce). Backups are uploaded to your Google Drive.
- **MEGA:** connect with MEGA credentials. Backups are uploaded to MEGA.
- **Run now:** trigger an immediate backup.
- **Restore from cloud:** download and restore a cloud backup.
- **Progress:** shows upload/download progress.
- **Logs:** view cloud backup history.

16.4 Backup Best Practices

- Keep at least 3 copies of your data (local backup, cloud backup, and one external).
- Test restoring a backup at least once a month.
- Before any major operation (year-end close, big import), create a manual backup.
- Never share your backup files - they contain all your financial data.

17. Software Updates

17.1 How Auto-Update Works

- The desktop app checks for updates **30 seconds after launch**, then every **4 hours**.
- It calls the server's update feed (<https://jindapos.com/updates/latest.yml>).
- If a newer version exists (respecting rollout %), it **downloads automatically** in the background.
- The download is verified by **SHA512 hash** before being marked complete.
- When the download is finished, the **Update Banner** appears in the app.
- Only an **admin** user can install the update (staff cannot).
- Before installing, the app **creates a pre-update backup** automatically.

17.2 Installing an Update

- 1 The Update Banner shows "Update available - version X".
- 2 The admin clicks **Install Update**.
- 3 The app creates a backup, then quits and installs.
- 4 Jinda restarts with the new version.

17.3 Publishing a New Version (Admin)

- 1 Build the new installer (`npm run dist:win`).
- 2 Upload **Jinda Setup.exe** to GitHub Releases (or R2).
- 3 Compute the SHA512 hash: `certutil -hashfile "Jinda Setup.exe" SHA512`.
- 4 Get the file size.
- 5 Go to `/admin/updates/create` on the server.
- 6 Fill in: version, release notes, download URL, file size, SHA512, rollout % (start with 25% for gradual), forced flag (usually false).
- 7 Set status to **published**.
- 8 Desktop apps will detect and download within 4 hours.

17.4 Rolling Back an Update

If a new version has a bug:

- 1 Go to `/admin/updates` and find the buggy version.
- 2 Change its status to **blocked** or **rollback**.
- 3 Re-publish the previous good version (or set the buggy one to rollback to point to the old one).
- 4 Users who haven't installed yet won't get the bad version. Users who did install can roll back manually.

17.5 Forced Updates

If a version has a critical security fix, set `forced: true`. Users must update before they can continue using the app.

18. Security Procedures

18.1 Security Overview

The platform is built with a "zero-trust" security model. Key protections:

Server:

- NextAuth for admin login (JWT, 24h session).
- Custom JWT + refresh-token rotation for POS users (reuse detection - if a stolen refresh token is reused, the whole token family is revoked).
- Account lockout after 5 failed logins (15-minute lockout).
- Device fingerprinting + device OTP step-up auth.
- Rate limiting (Redis-backed with memory fallback): default 10/min, license 5/min, auth 5/15min, API 60/min.
- Audit logging (29 event types, 3 severity levels).
- Security headers (CSP, HSTS in prod, X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy, Permissions-Policy).
- Strong password policy (min 8, upper/lower/digit/special).
- Anti-enumeration on forgot-password and verify-status (does not reveal if an email exists).
- License-device binding via HMAC-SHA256.

Desktop App:

- Electron security: `contextIsolation: true, nodeIntegration: false, sandbox: true, webSecurity: true`.
- No raw Node APIs exposed to the renderer - only a sanitized `electronSecureAPI` via `contextBridge`.
- Every IPC input is validated with Zod schemas.
- Every IPC handler can require auth, admin, and rate limiting.
- CSP headers (strict in production).
- URL allowlists for `shell.openExternal` and SaaS requests.
- SSRF protection (blocks requests to private IPs via DNS-rebinding checks).
- Database encrypted at rest (AES-256-CBC + HMAC).
- IntegrityService seals critical values (GST rates, user roles, period locks) - tamper-evident.
- Tamper-evident audit chain (verified on boot).
- Device ID protected by an encrypted persistent fallback (prevents cloning).
- Image uploads validated by magic bytes (PNG/JPEG) and size limits.
- Print HTML sanitized (no scripts, iframes, or event handlers).
- Tokens stored in OS keychain (keytar) with encrypted-file fallback.

18.2 Admin Security Duties

- **Regularly review** the Security Dashboard (`/admin/security`).
- **Rotate secrets** (NEXTAUTH_SECRET, LICENSE_BINDING_SECRET) if you suspect a breach. Note: rotating NEXTAUTH_SECRET invalidates all sessions.
- **Revoke licenses** of customers who chargeback or commit fraud.
- **Monitor** top failing IPs and suspicious users.
- **Update** the server and desktop app promptly when security patches are released.
- **Review audit logs** for unusual activity.

18.3 Password Policy

All passwords (admin and POS users) must be:

- At least 8 characters
- At least one uppercase letter

- At least one lowercase letter
- At least one digit
- At least one special character

18.4 Secret Management

- **Never** commit `.env`, `.env.local`, or any file containing real secrets to git.
- **Rotate** the `ENCRYPTION_PEPPER` only with a migration plan - changing it makes existing encrypted data unreadable.
- **Store** the `NEXTAUTH_SECRET` and `ENCRYPTION_PEPPER` in a password manager or secret vault.
- **Use** different secrets for dev and production.

18.5 What to Do If Compromised

- 1 Server breach:** Rotate `NEXTAUTH_SECRET` (invalidates all sessions). Rotate MongoDB password. Rotate SMTP password. Rotate AWS/R2 keys. Review audit logs. Revoke suspicious licenses.
- 2 Desktop app breach (single shop):** Revoke the license from the admin dashboard. The shop owner should create a new license. Change the local admin password.
- 3 Suspected data theft:** Restore from a known-good backup. Change all passwords. Notify affected customers if required by law.

19. Troubleshooting

19.1 Server Issues

Problem	Likely Cause	Fix
Website won't load	Server not running / Vercel deploy failed	Check <code>npm run dev</code> output; check Vercel build logs
Admin can't log in	Admin not seeded / wrong password	Run <code>npm run seed</code> ; or reset: <code>npx ts-node scripts/reset-admin.ts <new-password></code>
MongoDB connection error	Wrong URI / IP not whitelisted / DNS issue	Check <code>MONGODB_URI</code> ; add IP in Atlas Network Access; set <code>MONGODB_USE_GOOGLE_DNS=true</code>
Emails not sending	SMTP not configured	Set <code>SMTP_HOST/PORT/USER/PASS</code> (Gmail App Password); see <code>FIX-OTP-EMAIL-NOT-ARRIVING.md</code>
Email verification link points to wrong URL	<code>NEXTAUTH_URL</code> not set or <code>VERCEL_URL</code> used	Set <code>NEXTAUTH_URL=https://yourdomain.com</code> in Vercel env vars; see <code>FIX-EMAIL-VERIFICATION-URL.md</code>
License activation fails from app	App points to wrong server URL	Set <code>VITE_API_URL</code> correctly in desktop <code>.env</code> and rebuild; see <code>FIX-LICENSE-SERVER-URL.md</code>
Rate limit too strict	Redis or memory limiter	Adjust limits in <code>lib/rate-limit/rate-limit.ts</code>

Problem	Likely Cause	Fix
Duplicate users	Migration issue	Run <code>scripts/diagnose-duplicate-users.ts</code> then <code>scripts/cleanup-duplicate-users.ts</code>
Redis not connecting	Wrong URL / Redis not running	Check <code>REDIS_URL</code> ; start Redis; the app falls back to memory if Redis is down

19.2 Desktop App Issues

Problem	Likely Cause	Fix
App won't start	DB locked / encryption error / corrupt DB	Close other Jinda instances; restore from <code>backups/emergency_*.db</code> ; check <code>%APPDATA%\Jinda\logs</code>
<code>better-sqlite3</code> install fails	Missing build tools	Install Python 3 + VS Build Tools C++ workload; run <code>npm run postinstall</code>
License won't activate	Wrong key / device mismatch / server down	Check the key format; check internet; check <code>VITE_API_URL</code> ; try device transfer with password + OTP
"Unable to reach the license server"	App points to localhost in production	Set <code>VITE_API_URL=https://jindapos.com</code> in <code>.env</code> and rebuild
GST labels wrong on prints	Old templates	Rebuild the app (the <code>fix_print_*.py</code> scripts fixed this in source)
Print dialog shows Chromium preview	Wrong build	Rebuild; the native Windows dialog is forced via a command-line switch
Trial expired immediately	<code>.trial-lock</code> exists from a previous install	Intended behavior (prevents trial abuse); activate a license
Can't create staff user	Plan seat limit reached	Upgrade the plan or deactivate an existing staff user
App says "verification required"	Offline for >7 days	Connect to the internet so the app can verify the license
Database is locked	Another instance running	Close all Jinda instances; check Task Manager
Reports show wrong aging	Old code bug	Rebuild; <code>fix_aging.py</code> and <code>fix_report_aging.py</code> fixed this

19.3 Database Repair Scripts (Python)

These scripts fix or migrate the **live user database** at `%APPDATA%\Jinda\thisum_tseyig.db`. Run with Python 3. **Always back up the DB first.**

Script	What it does
<code>fix_pos.py</code>	Wires email-invoice into the POS post-sale flow
<code>fix_pos_gst.py</code>	Adds domestic GST support to the POS
<code>fix_db_tax.py</code>	Adds <code>tax_type</code> column to <code>invoices</code> table
<code>fix_engine_tax.py</code>	Adds <code>taxType</code> to the accounting engine
<code>fix_print_gst.py</code>	Makes GST labels dynamic on prints
<code>fix_print_pro_gst.py</code>	Same for the professional template
<code>fix_settings_gst.py</code>	Adds domestic GST rate to settings
<code>fix_aging.py</code>	Fixes aging bucket SQL (no negative days)
<code>fix_report_aging.py</code>	Fixes <code>julianday('now')</code> -> <code>julianday(date('now'))</code>
<code>fix_main.py</code>	Fixes the email-invoice IPC handler
<code>check_settings.py</code>	Prints all GST-related settings (diagnostic)
<code>migrate_po.py</code>	Adds <code>tax_type</code> to <code>purchase_orders</code> table
<code>migrate_po_selling_price.py</code>	Adds <code>selling_price</code> to <code>purchase_order_items</code> table

19.4 Useful Server Scripts

Run from the project root with `npx ts-node scripts/<name>`:

Script	What it does
<code>seed-admin.ts</code>	Create/update admin user (<code>npm run seed</code>)
<code>check-admin.ts</code>	Inspect the admin user
<code>reset-admin.ts <password></code>	Reset admin password
<code>fix-admin-role.ts</code>	Fix the admin role field
<code>migrate-device-verification.ts</code>	Add device-verification to existing users
<code>cleanup-duplicate-users.ts</code>	Remove duplicate PosUser records
<code>diagnose-duplicate-users.ts</code>	Diagnose duplicate users
<code>cleanup-update-files.ts</code>	Remove orphaned update files
<code>test-all.js</code>	End-to-end test runner
<code>test-email-tokens.js</code>	Test email token flow

Script	What it does
<code>test-license-api.js</code>	Test license API
<code>test-license-full.js</code>	Full license lifecycle test
<code>test-pos-connectivity.js</code>	Test POS↔server connectivity

19.5 Logs and Diagnostics

- **Desktop app logs:** Check `%APPDATA%\Jinda\logs` and the console output in dev mode.
- **Server logs:** Check the terminal (dev) or Vercel function logs (production).
- **Audit logs:** Both server (`AuditLog` in MongoDB) and desktop (`audit_logs` table) keep tamper-evident logs.
- **Health check:** `curl https://jindapos.com/api/health -> { "status": "ok" }`.

19.6 Getting Help

- Read the fix documents in `New folder\app\` (e.g., `FIX-DEVICE-OTP-EMAIL.md`).
- Read `BUILD_GUIDE.md`, `COMPLETION_SUMMARY.md`, `PROJECT_SUMMARY.md` in the desktop app folder.
- Read `ELECTRON-README.md`, `QUICK_START_ELECTRON.md`, `REDIS_SETUP.md`, `SECURITY_REMEDIATION.md`, `COMPREHENSIVE_SECURITY_BUGS_REPORT.md` in the project root.
- Read `COMPLETE_PROJECT_DOC.md` and `PROJECT_STRUCTURE.md` for the full architecture.

20. Maintenance Schedule

20.1 Daily

- The desktop app **auto-backs up** at 23:00 (verify it ran in Settings -> Backup).
- The desktop app **checks for updates** every 4 hours (verify users install them).
- The desktop app **verifies the license** once a day (silent).
- The server should be **monitored** (Vercel dashboard, MongoDB Atlas metrics).

20.2 Weekly

- **Review the Security Dashboard** (`/admin/security`) for alerts.
- **Review** the audit logs (server and desktop).
- **Check** for expiring licenses (`/admin/dashboard`) and contact customers to renew.
- **Test restoring** a desktop backup on a test machine.

20.3 Monthly

- **File GST returns** in the desktop app (GST page).
- **Close the accounting period** (Settings -> Period Close) after finalizing the month.
- **Review** the customer list for inactive accounts.
- **Update** software if a new version is available.
- **Rotate** API keys/secrets if your security policy requires it.

20.4 Quarterly

- **Review** all admin and POS user access. Remove inactive users.
- **Audit** license usage vs. plan limits.
- **Test** the disaster recovery plan (restore server from backup, restore desktop from cloud).
- **Review** Redis memory usage (if used).

20.5 Yearly

- **Rotate** `NEXTAUTH_SECRET` and `LICENSE_BINDING_SECRET` (with a migration plan - this invalidates sessions).
- **Review** and update this SOP document.
- **Archive** old audit logs and backups per your retention policy.
- **Renew** domain, SSL, MongoDB, Redis, and hosting subscriptions.

Appendix A - Command Reference

A.1 Server Commands (run in Jinda POS)

```
npm install          # Install dependencies
npm run dev          # Start dev server (http://127.0.0.1:3000)
npm run build        # Production build
npm run start        # Start production server
npm run lint         # Lint
npm run seed         # Create/update admin user
npm run electron:dev # Electron admin wrapper (dev)
npm run electron:build:win # Build Windows admin installer
```

A.2 Desktop App Commands (run in Jinda POS\New folder\app)

```
npm install          # Install + rebuild native modules
npm run dev          # Full dev (Vite + Electron watcher + launch)
npm run dev:win      # Same (Windows)
npm run dev:compile  # Compile Electron TS once
npm run dev:electron # Launch Electron only
npm run build        # Build React renderer (vite build)
npm run lint         # Lint
npm run preview      # Preview built renderer
npm run electron:dev # Build + compile + run Electron (dev)
npm run electron:start # Build + compile + run Electron (prod)
npm run electron:compile # Compile Electron TS to dist-electron/
npm run dist         # Build installers (all platforms)
npm run dist:win     # Build Windows installer + portable
npm run dist:mac     # Build Mac DMG + ZIP
npm run dist:linux   # Build Linux AppImage + DEB + RPM
npm test            # Run test suite
```

A.3 Useful One-Off Commands

```
# Reset admin password
npx ts-node scripts/reset-admin.ts <new-password>

# Check admin user
npx ts-node scripts/check-admin.ts

# Test email flow
node scripts/test-email-tokens.js

# Test license API
node scripts/test-license-api.js

# Full license lifecycle test
node scripts/test-license-full.js

# Test POS↔server connectivity
node scripts/test-pos-connectivity.js

# Compute SHA512 for an update file (Windows)
certutil -hashfile "Jinda Setup.exe" SHA512

# Start local Redis (Docker)
docker run -d --name redis -p 6379:6379 redis:7-alpine

# Generate a random secret
openssl rand -base64 32      # for NEXTAUTH_SECRET
openssl rand -hex 32        # for ENCRYPTION_PEPPER
```

A.4 Build Targets

Platform	Command	Output
Windows	<code>npm run dist:win</code>	release/Jinda Setup.exe, release/DhisumTseyig-Portable.exe
macOS	<code>npm run dist:mac</code>	release/Jinda-<version>.dmg, .zip
Linux	<code>npm run dist:linux</code>	release/*.AppImage, *.deb, *.rpm

Appendix B - File Structure

B.1 Server (Root: Jinda POS)

Server

Jinda POS

Jinda POS/

└─	app/	Next.js App Router
└─	page.tsx	Landing page

		layout.tsx	Root layout (Navbar, Footer, ChatWidget)
		about/, contact/, docs/, download/, features/, license/,	
		license-activate/, pricing/, privacy/, refund/, reset-password/,	
		security/, terms/, updates/, verify-email	
		admin/	Admin dashboard
		login/, dashboard/, settings/, security/, messages	
		licenses/ (list, create, [id])	
		customers/ (list, [id], [id]/edit)	
		updates/ (list, create, edit/[id])	
		api/	40 API route files (see Section 10.10)
		auth/ (nextauth, register, login, logout, refresh, status,	
		verify-email, verify-device, resend-*, verification-status	
		forgot-password, reset-password, change-password, captcha)	
		license/ (verify, activate, revoke)	
		admin/ (licenses, customers, users, messages, security)	
		updates/ (route, [id], latest, upload)	
		chat/, contact/, download/, health/, stats/, terms/, privacy	
		components/	UI components (layout, providers, chatbot, seo, admin, ui)
		lib/	
		db/mongodb.ts/	MongoDB connection (singleton)
		models/	11 models (Admin, PosUser, License, Customer, Session, Token, Trial, Update, AuditLog, ContactMessage)
		auth/	auth.config.ts, auth.middleware.ts, device.ts, jwt.ts, tokens.ts
		email/email.ts/	Nodemailer
		rate-limit/	Redis + memory fallback
		audit/auditLogger.ts/	

└─	chatbot/system-prompt.ts/	
└─	s3/s3-client.ts/	
└─	data/bhutanLocations.ts/	
└─	validation/schemas.ts/	Zod schemas
└─	utils.ts	cn(), generateLicenseKey(), etc.
└─	scripts/	15 scripts (seed, check, reset, fix, migrate, test)
└─	middleware.ts	Security & SEO headers
└─	electron/ (intended; may be absent — see ELECTRON-README.md)	
└─	dist-electron/	Build output for admin Electron app
└─	public/	Static assets
└─	.env.example, .env.local	Environment files
└─	package.json	site-jinda
└─	Docs: COMPLETE_PROJECT_DOC.md, PROJECT_STRUCTURE.md, ELECTRON-README.md QUICK_START_ELECTRON.md, REDIS_SETUP.md, SECURITY_REMEDIATION.md/ COMPREHENSIVE_SECURITY_BUGS_REPORT.md, SOP.md (this file)/	

B.2 Desktop App (Jinda POS\New folder\app)

Desktop App

Jinda POS\New folder\app

New folder/app/		
└─	src/	
└─	App.tsx	Main app + routing state machine
└─	main.tsx	React entry
└─	components/	
└─	ui/	40+ shadcn/ui components
└─	auth/AgreementOverlay.tsx/	EULA gate

			Header.tsx, Sidebar.tsx, Logo.tsx, Loader.tsx	
			Notification.tsx, PrintPreviewModal.tsx	
			UpdateBanner.tsx, TrialBanner.tsx	
			config/api.ts/	SaaS API URL/endpoints
			data/bhutanLocations.ts/	
			database/DatabaseManager.ts/	SQLite schema, encryption, migrations
			hooks/	use-app-version, use-mobile
			lib/utlis.ts/	
			pages/	29 pages (Welcome, LicenseActivation, SetupAccount, CheckEmail, VerifyEmail, ForgotPassword, DeviceVerification, Login, Dashboard, POS, Inventory, Contacts, Transactions, PurchaseOrder, Quotation, Refund, CustomerStatement, Expense, Recurring, AgedReports, TieredPricing, GST, Reports, Employee, Branch, Barcode, ImportExport, AuditLogViewer, Settings)
			services/	32 services (AccountingEngine, Accounting, Inventory, GST, Report, Printing, Backup, CloudBackup, License, Update, HeldCart, Expense, PurchaseOrder, Audit, Quotation, Refund, Recurring, AgedReport, Export, Barcode, Employee, Branch, CSVImport, SplitPayment, TieredPricing, EmailInvoice, Payroll, Integrity, etc.)
			store/appStore.ts/	Zustand global store
			types/index.ts/	All TypeScript types
			utlis/	audio, deviceId, electronApi, encryption, installSecret, internetCheck, licenseStorage, processPolyfill, secureStorage
			electron/	
			main.ts	3250 lines: window, IPC, DB, services, security
			preload-secure.ts	1371 lines: secure contextBridge API
			update-manager.ts	334 lines: electron-updater wrapper
			security/	validators (Zod), SecureEncryption, guides
			dist-electron/	Compiled JS (generated)
			scripts/tests/	Custom test runner
			build/	Build resources (icon.ico, license.txt)
			release/	Build output (installers)
			public/	Static assets

└─	.env, .env.example	
└─	package.json	dhisum-tseyig
└─	vite.config.ts, tsconfig*.json, tailwind.config.js, postcss.config.js	
└─	electron-builder-simple.json	Quick-test build config
└─	Employees.xlsx, Transactions.xlsx	Sample import files
└─	fix_*.py	13 Python fix/migration scripts
└─	fix-balances.cjs/.js/	
└─	Docs: README.md, BUILD_GUIDE.md, COMPLETION_SUMMARY.md	
	PROJECT_SUMMARY.md, info.md, FIX-DEVICE-OTP-EMAIL.md/	
	FIX-EMAIL-VERIFICATION-URL. md, FIX-LICENSE-SERVER-URL.md/	
	FIX-OTP-EMAIL-NOT-ARRIVING. md/	

Appendix C - Default Credentials and Locations

C.1 Default Credentials

What	Default	Where to change
Server admin	Set by ADMIN_USERNAME / ADMIN_PASSWORD in .env.local	npm run seed or scripts/reset-admin.ts
Desktop app initial admin	User chooses on first run	Settings -> Users (after setup)
Common test default	admin / admin123	Change immediately in production
MongoDB	Set in Atlas	Atlas Database Access
SMTP (Gmail)	Gmail address + App Password	.env.local SMTP_USER / SMTP_PASS

Always change default passwords before going live.

C.2 Important File Locations

Server (project root):

File	Purpose
<code>.env.example</code>	Template for env vars (safe to share)
<code>.env.local</code>	Real env vars (never commit)
<code>middleware.ts</code>	Security & SEO headers
<code>lib/db/mongodb.ts</code>	MongoDB connection
<code>lib/models/</code>	Database models
<code>lib/auth/</code>	Authentication logic
<code>lib/email/email.ts</code>	Email sending
<code>lib/rate-limit/rate-limit.ts</code>	Rate limiting
<code>lib/utls.ts</code>	License key generation
<code>scripts/seed-admin.ts</code>	Admin seeder

Desktop App (New folder/app):

File	Purpose
<code>.env</code>	Desktop env vars (<code>VITE_API_URL</code> , <code>ENCRYPTION_PEPPER</code>)
<code>electron/main.ts</code>	Main process (window, IPC, DB, services)
<code>electron/preload-secure.ts</code>	Preload (secure API bridge)
<code>electron/update-manager.ts</code>	Auto-updater
<code>src/database/DatabaseManager.ts</code>	SQLite schema & encryption
<code>src/services/LicenseService.ts</code>	License logic
<code>src/services/PrintingService.ts</code>	Invoice/receipt templates
<code>src/services/ExportService.ts</code>	Excel/PDF export
<code>src/services/AccountingEngineService.ts</code>	Double-entry engine
<code>src/store/appStore.ts</code>	Zustand store
<code>src/config/api.ts</code>	SaaS API config
<code>build/icon.ico</code>	Windows icon
<code>build/license.txt</code>	NSIS license text

C.3 Runtime Locations (End-User's Computer)

Path	What
%APPDATA%\Jinda\dhisum_tseyig.db	The SQLite database (plaintext while running)
%APPDATA%\Jinda\dhisum_tseyig.db.secure	The encrypted database (while closed)
%APPDATA%\Jinda\license.json	License info
%APPDATA%\Jinda\trial.json	Trial info
%APPDATA%\Jinda\trial-lock	Encrypted trial lock
%APPDATA%\Jinda\install-key	Per-install encryption secret
%APPDATA%\Jinda\device-id	Encrypted device ID
%APPDATA%\Jinda\pos-session.enc	Encrypted POS session
%APPDATA%\Jinda\secure-tokens	Encrypted token fallback
%APPDATA%\Jinda\backups\	Local backups
%APPDATA%\Jinda\backups\emergency_*.db	Emergency backups
%APPDATA%\Jinda\cache\	Cache dir
%APPDATA%\Jinda\license-agreement.json	EULA acceptance
OS keychain (Jinda-POS / saas-tokens)	POS tokens (keytar)

On macOS, replace %APPDATA%\Jinda with ~/Library/Application Support/Jinda. On Linux, ~/.config/Jinda.

C.4 Build Output

Path	What
New folder/app/dist/	Built React renderer
New folder/app/dist-electron/	Compiled Electron JS
New folder/app/release/Jinda Setup.exe	Windows NSIS installer
New folder/app/release/DhisumTseyig-Portable.exe	Windows portable
New folder/app/release/win-unpacked/Jinda.exe	Unpacked app
New folder/app/release-test/	Quick-test build output

Appendix D - Glossary

Term	Meaning
API	A web address a program calls to get or send data
AppId	A unique ID for the app (<code>com.jinda.app</code>) used by Windows/Mac
Audit log	A tamper-evident record of who did what
bcrypt	A strong password-hashing algorithm
Cloudflare R2	Cloud file storage (like AWS S3)
Context isolation	Electron security: separates the renderer from Node.js
CSP	Content Security Policy: rules about what code can run
EULA	End-User License Agreement
Electron	A tool that turns web code into a desktop program
electron-builder	Packages an Electron app into an installer
electron-updater	Auto-updates Electron apps
GST	Goods and Services Tax (5% in Bhutan)
HMAC	A secure way to verify data hasn't been changed
IPC	Inter-Process Communication (renderer ↔ main process)
JWT	JSON Web Token (a signed login token)
MongoDB	An online database (used by the server)
Next.js	A React website framework
NextAuth	Login library for Next.js
NSIS	The Windows installer format
OTP	One-Time Password (6-digit code sent by email)
POS	Point of Sale (the sales counter)
Preload script	The bridge that safely exposes main-process functions to the renderer
Rate limiting	Stopping someone from making too many requests
Redis	A fast in-memory cache (for rate limiting)
Refresh token	A long-lived token used to get new short-lived access tokens
SaaS	Software as a Service (online account system)
SHA512	A hash used to verify file integrity
SOP	Standard Operating Procedure (this document)

Term	Meaning
SQLite	A database stored as a single file (used by the desktop app)
SSRF	Server-Side Request Forgery (an attack we block)
Vercel	A hosting service for Next.js websites
Vite	A fast build tool for React
VITE_API_URL	The server URL the desktop app talks to
Zod	A library for validating data shapes
Zustand	A small state-management library for React

End of SOP Document

This document covers the complete operation of the Jinda POS Platform - both the Next.js licensing server and the Electron desktop POS/accounting app - from installation through daily use, security, backups, updates, and troubleshooting. Keep it updated as the software evolves.